Agentes Inteligentes (AIN)

Práctica 2: CrewAI

CrewAI

Objetivos

- Conocer herramientas de desarrollo de agentes basados en modelos de lenguaje (LLM)
- Aprender a construir equipos de agentes de este tipo que trabajan juntos para abordar tareas complejas.

¿Qué son Agentes LLM?

- Los **agentes LLM (Large Language Model)** son sistemas inteligentes impulsados por modelos de lenguaje.
  - Realizan tareas específicas, interactuando con otros agentes y humanos
  - Aprenden del contexto para abordar problemas complejos de manera colaborativa.
  - Capacidad para interpretar instrucciones en lenguaje natural, razonar sobre ellas, y producir respuestas coherentes y útiles.

Ejemplos:

- Investigación en profundidad sobre temas específicos.
- Automatización de flujos de trabajo complejos.
- Asistentes virtuales interactivos y colaborativos.

CrewAI

Frameworks existentes para desarrollo de Agentes LLM:

- Facilitan la creación, gestión y despliegue de equipos colaborativos de agentes.
- Permiten definir roles específicos, objetivos y flujos de interacción entre agentes.

Ejemplos: Autogen, LangGraph, ADK, CrewAI, …

CrewAI

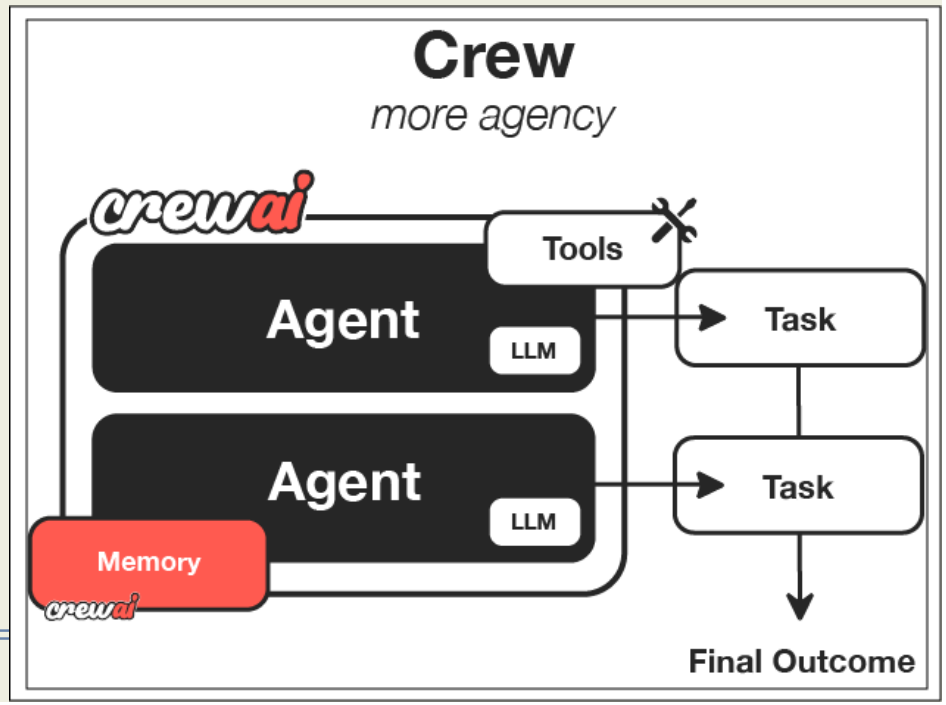
¿Qué es?

CrewAI es un framework en Python que permite crear equipos de agentes autónomos que trabajan juntos para alcanzar un objetivo.

Cada agente en CrewAI es un experto en una tarea específica (como investigar, redactar, planificar o analizar) y todos colaboran como un equipo para tareas como:

- 📧 redactar un correo personalizado
- 📈 generar un plan de negocio
- 📚 escribir un libro
- 📝 crear una entrada de blog

https://docs.crewai.com/



CrewAI

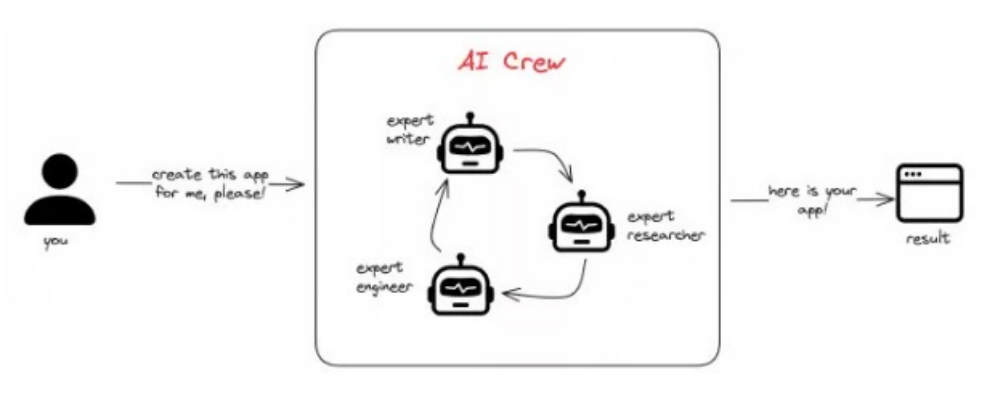
¿Qué es?

Conceptos clave:

- **Agentes:** son miembros especializados de un equipo que asumen roles concretos (investigador, redactor), utilizan **herramientas** específicas, pueden delegar **tareas** y toman decisiones de manera autónoma.

- **Crews**: Permite definir equipos de agentes especializados con roles, herramientas y objetivos propios, optimizados para la autonomía y la colaboración.

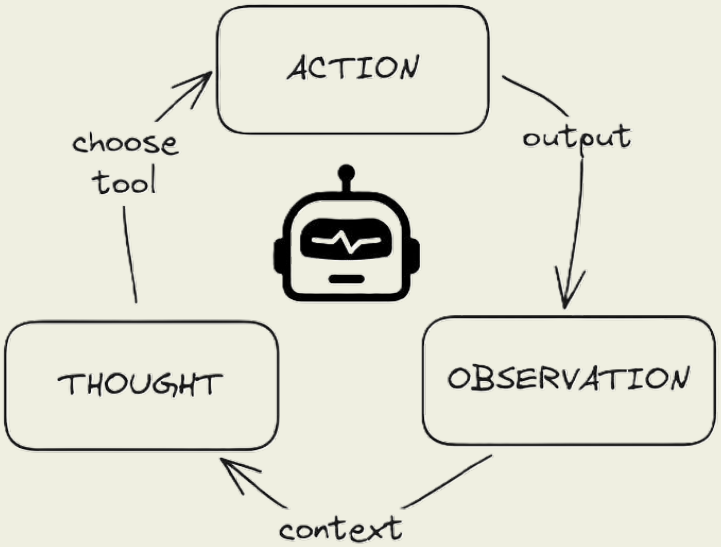
- **Flows**: Permite orquestar tareas con control granular, llamadas a LLM individuales y flujos nativos entre agentes.



CrewAI

¿Qué es?

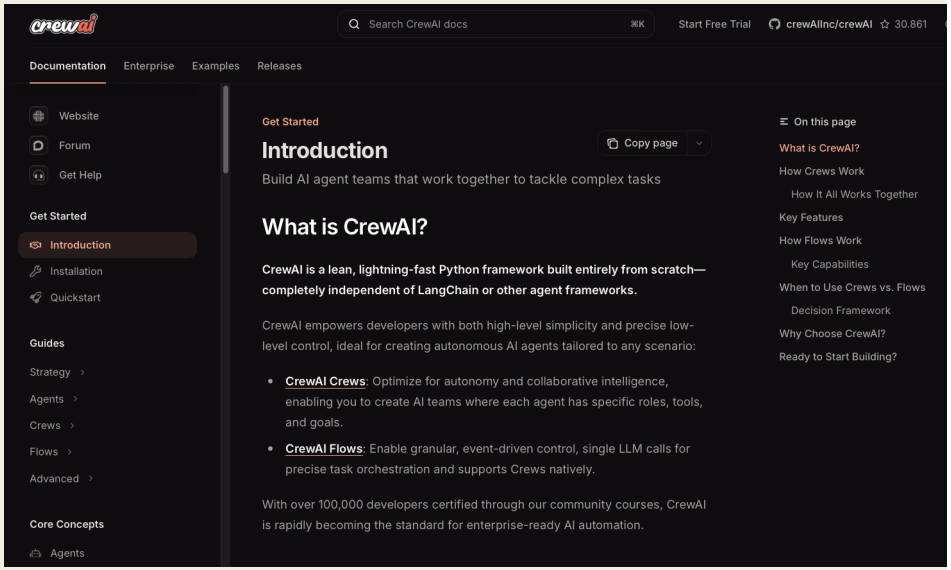
- Los agentes en CrewAI son esencialmente **agentes ReAct**
- Son, en cierto sentido, autónomos. Esto significa que pueden razonar por sí mismos y completar sus tareas sin necesidad de indicarles explícitamente qué pasos deben seguir.
- Solo necesitan una instrucción inicial (prompt) que les indique lo que esperamos de ellos. En este caso, esperamos que realicen la tarea que les hemos asignado.



CrewAI

Documentación

- Documentación: https://docs.crewai.com/introduction
- Github: https://github.com/crewAIInc/crewAI
- Github ejemplos: https://github.com/crewAIInc/crewAI-examples



CrewAI

Instalación

1️⃣ **Crear entorno Conda compatible**
CrewAI requiere **Python >= 3.10 y < 3.13**, así que:

```
conda create -n crewai-env python=3.12
conda activate crewai-env
```

2️⃣ **Instalar CrewAI con pip**
Aunque CrewAI recomienda uv, puedes intentar instalarlo con pip si no necesitas funcionalidades avanzadas del CLI:

```
pip install crewai
```

3️⃣ **Verificar instalación (si funcionó con pip)**

```
crewai --help
```

CrewAI

Definir LLM

Los modelos de lenguaje a gran escala son la inteligencia central detrás de los agentes de CrewAI. Permiten que los agentes comprendan el contexto, tomen decisiones y generen respuestas similares a las humanas.

Configuración:

```
from crewai import LLM

llm = LLM(
   model = ”nombre del modelo",
   base_url=”http:// ….",
   api_key = ”XXXXXXXX",
)

#Uso de modelos en la UPV
llm = LLM(
   model = "openai/qwen-mini"
   base_url="https://api.poligpt.upv.es/",
   api_key = "sk-LFXs1kjaSxtEDgOMlPUOpA",
)
```

Opción UPV
- URL: `https://api.poligpt.upv.es/`
- API Key: `sk-LFXs1kjaSxtEDgOMlPUOpA`

Con VPN fuera de la UPV

Otras opciones:
- Local: Ollama
- Online: Groq (gratuito), OpenAI (pago)

Otros modelos disponibles: "openai/qwen3:4b", “openai/deepseek-r1:7b", "openai/llama", "openai/gemma"

CrewAI

Crear Agentes

Los agentes en CrewAI se configuran con una serie de parámetros que definen su comportamiento, objetivos y personalidad:

- **Role**. Define el **rol funcional** del agente dentro del equipo.
- **Goal**. Especifica el **objetivo principal** que debe cumplir el agente.
- **Backstory**. Proporciona un contexto o trasfondo que influye en el estilo de interacción y razonamiento del agente.
- **Llm**. Indica qué **modelo de lenguaje** se utilizará como motor cognitivo del agente.
- **Verbose**. Controla el nivel de **registro de información (logging)** durante la ejecución.

```
research_agent = Agent(
      role=”Investigador analista",
      goal=”Encuentra y resume información sobre temas específicos",
      backstory=" Eres un investigador con experiencia y gran atención al detalle",
      llm = llm,
      verbose=True # Habilitar logging para depuración)
```

Hay muchos más parámetros: https://docs.crewai.com/concepts/agents

CrewAI

Crear Tareas

Una **Tarea (Task)** es una asignación específica que realiza un **Agente**.

Las tareas incluyen todos los detalles necesarios para su ejecución: descripción, agente responsable, herramientas requeridas, salida esperada…

Las tareas pueden ejecutarse de dos formas:

- **Secuencial**: Las tareas se ejecutan en el orden en que han sido definidas.
- **Jerárquica**: Las tareas se asignan a los agentes según sus roles y áreas de especialización.

```
from crewai import Task

research_task = Task(
    description="""
       Realiza una investigación exhaustiva sobre los agentes de inteligencia artificial.
       Asegúrate de encontrar información interesante y relevante,
       teniendo en cuenta que el año actual es 2025.    """,
    expected_output="""
       Una lista con 10 puntos clave sobre la información más relevante acerca de los
agentes de inteligencia artificial.
    """,
    agent=research_agent
)
```

Hay muchos más parámetros: https://docs.crewai.com/concepts/tasks

CrewAI

Crear Crew

- Una crew en CrewAI representa un grupo colaborativo de agentes que trabajan juntos para completar un conjunto de tareas.
- Cada crew define la estrategia de ejecución de tareas, la colaboración entre agentes y el flujo de trabajo general.

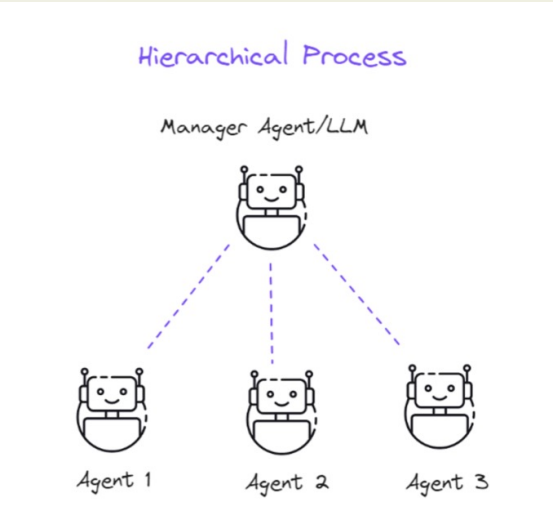
```
crew = Crew(
    agents=[research_agent, writer_agent],
    tasks=[research_task, write_article_task],
    verbose=True
)

result = crew.kickoff(inputs={"topic": ”Sistemas multi-agente"})

print(result.raw)
```

CrewAI

Crear Crew

- **Proceso de Ejecución (process)**
  - **Proceso Secuencial:** Las tareas se ejecutan una tras otra, lo que permite un flujo de trabajo lineal.
  - **Proceso Jerárquico:** Un agente gestor coordina al equipo, delega tareas y valida los resultados antes de continuar. Se requiere un *manager_llm* o *manager_agent* para este proceso, y su participación es necesaria para validar el flujo del proceso.

```
crew = Crew(
  agents=my_agents,
  tasks=my_tasks,
  process=Process.sequential )
```




```
crew = Crew(
  agents=my_agents,
  tasks=my_tasks,
  process=Process.hierarchical,
  manager_agent=my_manager_agen
  # or
  # manager_llm="gpt-4o" )
```

CrewAI

Añadir Herramientas

Una **herramienta (tool)** es una habilidad o función que los agentes pueden utilizar para llevar a cabo diversas acciones. Por ejemplo, una búsqueda en internet.

Usos comunes de herramientas:

- Consultas a APIs externas
- Cálculos o análisis específicos
- Acceso a bases de datos o documentos
- Operaciones personalizadas

Es necesario instalar un plugin: *pip install 'crewai[tools]'*

- Existen multitud de tools predefinidas: https://docs.crewai.com/concepts/tools
- Se pueden crear nuevas tools

CrewAI

Añadir Herramientas

Ejemplo de uso de **SerperDevTool (motor de búsqueda)**

```
from crewai_tools import SerperDevTool

os.environ['SERPER_API_KEY'] = ”xxxxxxxxx”  # es necesario darse de alta y obtener la key

searching_tool = SerperDevTool()

planner = Agent(
    role=”Buscador de informacion",
    goal="Buscas contenido científico riguroso y atractivo sobre {topic}",
    backstory="""”Eres un experto en buscar información sobre el tema: {topic}.
                Utilizas herramientas avanzadas de búsqueda para recopilar información
""",
    allow_delegation=False,
    tools=[searching_tool],
    verbose=True,
    llm=llm
)
```

Obtener la API Key de Serper en https://serper.dev

Añadir Herramientas

Cómo crear tu propia Tool

```
from crewai.tools import tool

@tool("Nombre de mi herramienta")
def my_tool(question: str) -> str:
    """Descripción clara de lo que hace esta herramienta.
    El agente necesita esta información para saber cuándo y cómo usarla."""

    # Aquí va la lógica de la herramienta
    return "Resultado de tu herramienta personalizada”
```

Donde:

- **@tool("Nombre")**: Decorador que registra la función como una herramienta accesible para los agentes.
- **def my_tool(...)**: Define la lógica específica que ejecutará tu herramienta.
- **"""Docstring"""**: Describe qué hace la herramienta. Esta descripción es fundamental para que los agentes puedan decidir cuándo utilizarla.
- **return**: Devuelve el resultado que la herramienta generará para el agente.

Añadir Herramientas

Ejemplo de custom tool

```
from crewai.tools import tool

@tool("LeerArchivo")
def leer_archivo(ruta: str) -> str:
    """Devuelve el contenido completo de un archivo de texto dado su ruta."""
    try:
        with open(ruta, 'r', encoding='utf-8') as archivo:
            contenido = archivo.read()
        return contenido
    except Exception as e:
        return f"Error al leer el archivo: {e}"
```

CrewAI

Crear Flujos

Los flujos son secuencias estructuradas de tareas que permiten coordinar y automatizar procesos dentro de un equipo de agentes.
Cada flujo define el orden en que se ejecutan las tareas y las condiciones bajo las cuales se activan, facilitando así la colaboración eficiente entre agentes para alcanzar objetivos específicos.

Elementos:

1. Flow (Clase base): Es la estructura principal que define el flujo. Se hereda para crear flujos personalizados.

2. @start(): Decorador que indica el método inicial del flujo. Es el primer paso que se ejecuta cuando se llama al flujo

   @start() def inicio(self): return "Paso inicial"

3. @listen(<otro_método>): Decorador que conecta un método con otro anterior. El método decorado se ejecuta después de recibir la salida del método indicado.

   @listen(inicio) def paso_dos(self, salida_anterior): return f"Recibido: {salida_anterior}"

4. kickoff(): Método que inicia la ejecución del flujo completo. Ejecuta el método con @start() y luego los que lo escuchen con @listen(...).

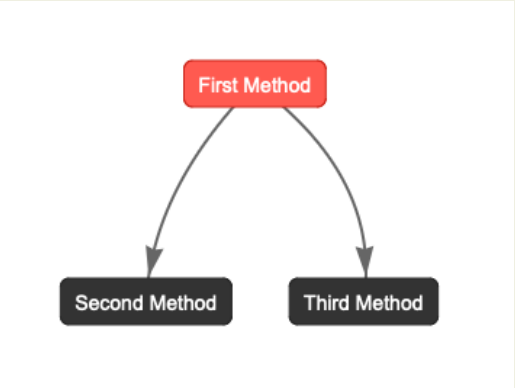
   flujo = MiFlujoPersonalizado() resultado = flujo.kickoff()

CrewAI

Crear Flujos

Ejemplo:

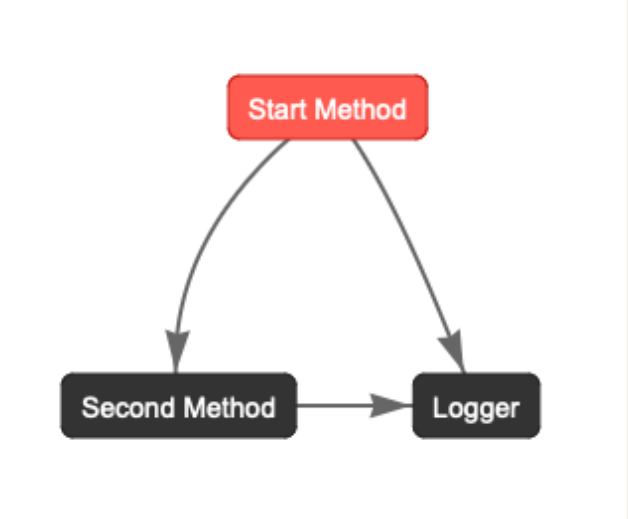
```
from crewai.flow.flow import Flow, listen, start

class OutputExampleFlow(Flow):
    @start()
    def first_method(self):
        return "Output from first_method"

    @listen(first_method)
    def second_method(self, first_output):
        return f"Second method received: {first_output}”

    @listen(first_method)
    def third_method(self, first_output):
        return f"Third method received: {first_output}"

flow = OutputExampleFlow()
final_output = flow.kickoff()

print("---- Final Output ----")
print(final_output)
```



Lógica condicional: or

- La función or_ en los Flows permite **escuchar múltiples métodos** y ejecutar el método receptor **cuando cualquiera** de los métodos especificados genere una salida.

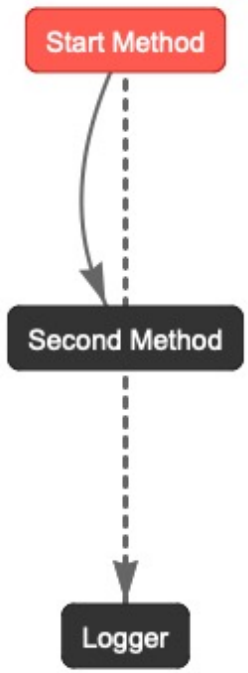
```
from crewai.flow.flow import Flow, listen, or_, start

class OrExampleFlow(Flow):

    @start()
    def start_method(self):
        return "Hello from the start method"

    @listen(start_method)
    def second_method(self):
        return "Hello from the second method"

    @listen(or_(start_method, second_method))
    def logger(self, result):
        print(f"Logger: {result}")

flow = OrExampleFlow()
flow.kickoff()
```



Lógica condicional: and

- La función and_ en los Flows permite **escuchar múltiples métodos** y ejecutar el método receptor **solo cuando todos** los métodos especificados hayan generado una salida.

```
from crewai.flow.flow import Flow, and_, listen, start

class AndExampleFlow(Flow):

    @start()
    def start_method(self):
        self.state["greeting"] = "Hello from the start method"

    @listen(start_method)
    def second_method(self):
        self.state["joke"] = "What do computers eat? Microchips."

    @listen(and_(start_method, second_method))
    def logger(self):
        print("---- Logger ----")
        print(self.state)

flow = AndExampleFlow()
flow.kickoff()
```



Enrutador (Router)

- El decorador @router() en los Flows permite definir una **lógica de enrutamiento condicional** basada en la salida de un método.
  Puedes especificar **rutas diferentes** según el resultado del método, lo que te permite **controlar dinámicamente el flujo de ejecución**.

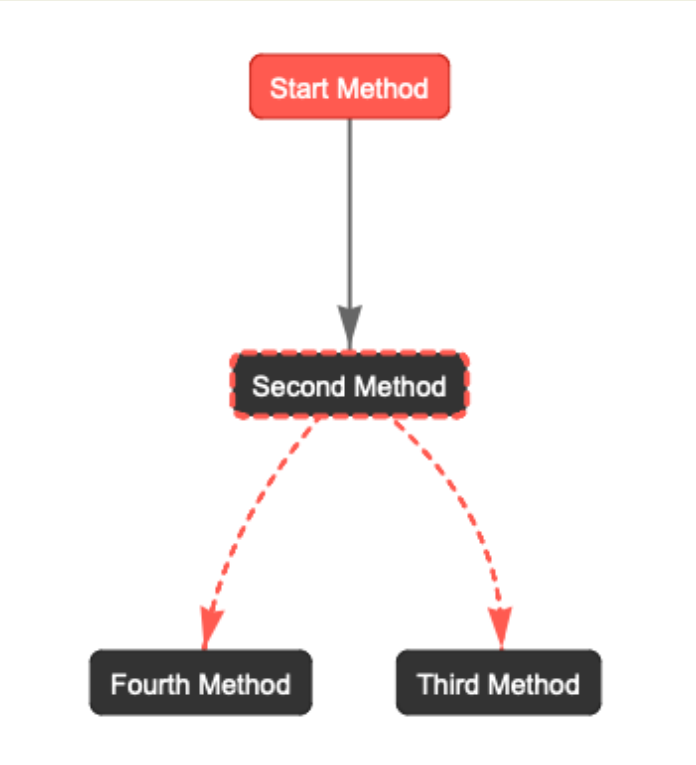
```
import random
from crewai.flow.flow import Flow, listen, router, start
from pydantic import BaseModel

class ExampleState(BaseModel):
    success_flag: bool = False

class RouterFlow(Flow[ExampleState]):

    @start()
    def start_method(self):
        print("Starting the structured flow")
        random_boolean = random.choice([True, False])
        self.state.success_flag = random_boolean
```

```
@router(start_method)
def second_method(self):
    if self.state.success_flag:
        return "success"
    else:
        return "failed”

@listen("success")
def third_method(self):
    print("Third method running")

@listen("failed")
def fourth_method(self):
    print("Fourth method running")
```

```
flow = RouterFlow()
flow.kickoff()
```

Enrutador (Router)

- El decorador @router() en los Flows permite definir una **lógica de enrutamiento condicional** basada en la salida de un método.
  Puedes especificar **rutas diferen** e te permite **controlar dinámicamente el flujo de ejecu**




```
import random
from crewai.flow.flow import F
from pydantic import BaseModel

class ExampleState(BaseModel):
    success_flag: bool = False

class RouterFlow(Flow[ExampleS

    @start()
    def start_method(self):
        print("Starting the st
        random_boolean = rando
        self.state.success_fla
```

```
er(start_method)
second_method(self):
f self.state.success_flag:
    return "success"
else:
    return "failed"

en("success")
third_method(self):
print("Third method running")

en("failed")
fourth_method(self):
print("Fourth method running")

flow = RouterFlow()
flow.kickoff()
```

CrewAI

Ejemplos

Ejemplo de un agente LLM que actúa como redactor científico para generar un artículo estructurado sobre un tema específico

- Ver código “ejemplo_agente.py”

Ejemplo de un agente redactor de un blog sobre un tema específico, utilizando herramientas externas como SerperDevTool para realizar búsquedas web que enriquecen el contenido generado

- Ver código “ejemplo_tools.py”

Ejemplo de flujo que genera un poema con un numero aleatorio de versos usando un agente, y guarda el resultado en un archivo

- Ver código “ejemplo_flujo_poema.py”

Ejercicio Práctico: *Desarrollo de un Sistema Multiagente con CrewAI*

Objetivo de la práctica

- Desarrollar un sistema en CrewAI que utilice un equipo de agentes, herramientas (*tools*) y un flujo personalizado para redactar un documento estructurado con referencias sobre un tema técnico o científico de libre elección.

Requisitos funcionales

- Diseñar un flujo personalizado (Flow) que defina el orden de ejecución de los agentes.
- Crear al menos 3 agentes con roles diferenciados (por ejemplo: investigador, redactor, verificador).
- Integrar al menos dos herramientas externas, como una API de búsqueda (Serper, SerpAPI) o lector de archivos.
- Generar un documento final estructurado que explique el sistema propuesto.
- El sistema debe poder recibir el tópico del documento como entrada (input).

Nota Máxima Práctica 2: 1.5 sobre la nota final de la asignatura (Práctica 1: 3.5)

CrewAI

Aspectos a tener en cuenta

1. Afinar la descripción de la tarea (Task.description)
    1. Debe ser clara, detallada y bien estructurada.
    2. Incluir pasos concretos que el agente debe seguir.
2. Definir correctamente los roles y objetivos del agente (Agent.role y Agent.goal)
    1. Cada agente debe tener un propósito bien diferenciado y el goal debe estar alineado con la tarea asignada.
3. Redactar un backstory coherente y útil
    1. Refuerza el comportamiento deseado del agente.
    2. Aporta contexto que influye en el estilo y enfoque de respuesta.
4. Seleccionar el LLM adecuado
    1. Modelos más grandes ofrecen mejor rendimiento en tareas complejas pero tardan más.
5. Usar herramientas relevantes (tools)
    1. Añadir solo las que el agente necesite.
    2. Asegurar que tengan descripciones claras (docstring) para que el agente entienda cuándo usarlas.
6. Poner expected_output de la tarea
    1. Define con precisión qué se espera que genere el agente.
    2. Puede incluir formato (ej. Markdown), longitud o estructura.
7. Activar verbose=True para depurar
    1. Muy útil durante desarrollo y pruebas.
8. Gestionar correctamente las entradas del sistema (inputs)
    1. Diseñar bien los inputs que necesita el sistema (crew.kickoff(inputs={...})).